Mid-Term Lab Examination — 8-bit Accumulator

University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

Mid-Term Lab Examination | Group C1 | Spring 2026

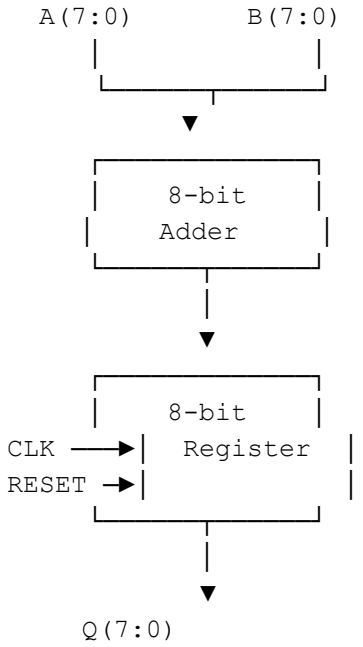
Duration: 1 hr Total Marks: 20

Student Name: Naima Tabassum Antora ID: 22201068

Mid-Term Task: 8-bit Accumulator

Objective. Design and implement an 8-bit accumulator in VHDL by combining the 8-bit adder and register concepts developed in the laboratory. The system must perform addition synchronously with the clock and store the result in an 8-bit register.

System Architecture



Functional requirement.

At every rising edge of CLK, the register stores the sum of the two 8-bit inputs. RESET has priority over the addition operation.

Required Operation

RESET	CLK	A, B	Operation / Q
1	↑	XX, XX	Q ← 00000000
0	↑	A, B	Q ← A + B
0	No rising edge	XX, XX	Q remains unchanged

Required Test Sequence

Your testbench must produce the following sequence. The expected Q value should be observed after the corresponding rising clock edge.

Step	RESET	A	B	Operation	Expected Q / Explanation
1	1	00000000	00000000	Reset	00000000 — Register is cleared
2	0	00000011	00000101	3 + 5	00001000 — Load sum
3	0	00000010	00000001	2 + 1	00000011 — Load new sum
4	0	11111111	00000001	255 + 1	00000000* — 8-bit overflow
5	0	10101010	00000101	170 + 5	10101111 — Load new sum

* Note: For Step 4, the mathematical result is 256, but an 8-bit output can represent only 0–255. Therefore, Q = 00000000 and the carry-out is discarded unless your implementation explicitly exposes it.

Implementation Requirements

- Use VHDL and IEEE.STD_LOGIC_1164.ALL.
- The register operation must be synchronized with the rising edge of CLK.
- RESET must be synchronous and active-high.
- Do not use a behavioral integer-based accumulator as a replacement for the required adder/register structure.
- The design should be modular and clearly identify the 8-bit adder and 8-bit register portions.
- The submitted testbench must generate a clock and apply the required input sequence.

Submission

While submitting to the classroom prepare the report from the third page of this template including the current template in the first and second page. From the third page write your report including but not limited to the following points.

- Attach your entity code and testbench code
- Attach your output visuals
- Challenges faced during the experiment
- Conclusion

Accumulator_8bit.v:

```
`timescale 1ns / 1ps

module accumulator_8bit(
    input [7:0] A,
    input [7:0] B,
    input CLK,
    input RESET,
    output reg [7:0] Q
);


// 8-bit Adder

wire [7:0] SUM;
assign SUM = A + B;


// 8-bit Register

always @(posedge CLK)

begin
    if (RESET)
        Q <= 8'b00000000;
    else
        Q <= SUM;
end


endmodule
```

Circuit Diagram / Schematic:

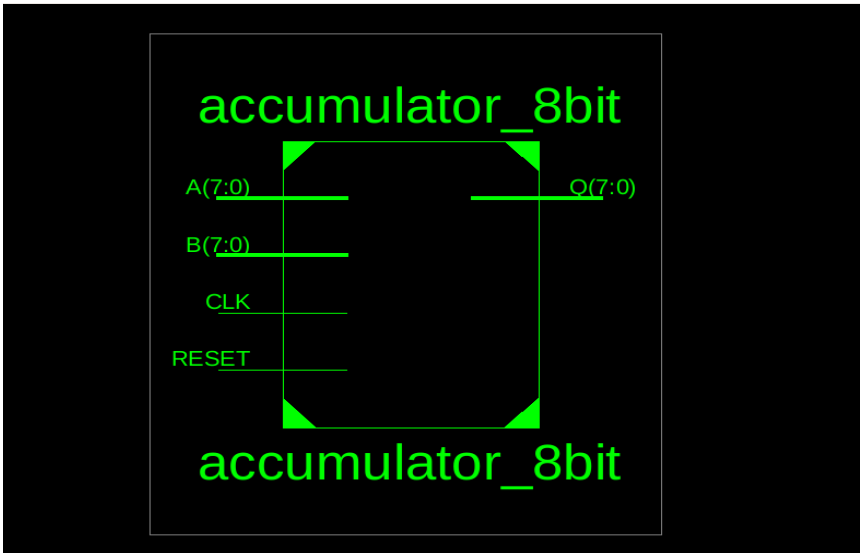


Fig. 1 (a): RTL Schematic of the 8-bit Accumulator (Top Level)

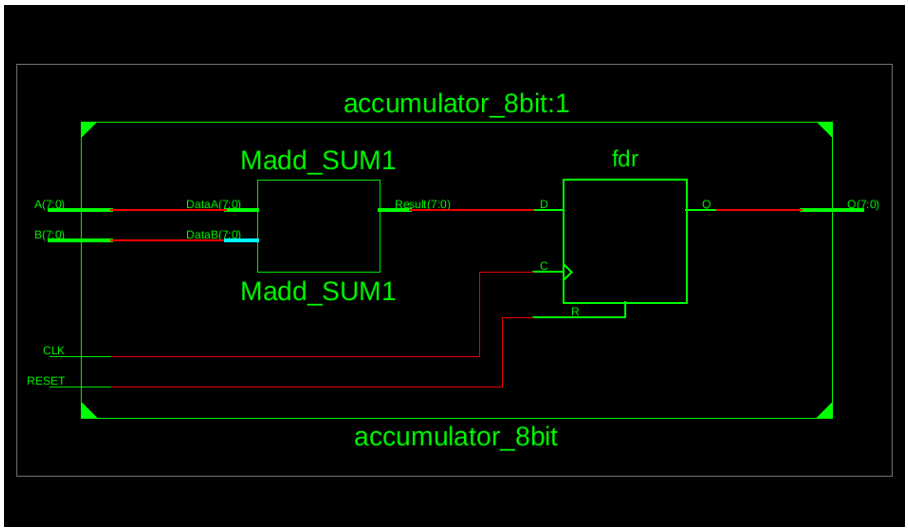


Fig. 1 (b): RTL Schematic of the 8-bit Accumulator (Expanded)

The 8-bit adder adds A and B to produce SUM and stores the result in SUM. Then, at the rising edge of the CLK, the 8-bit register stores SUM into Q. If RESET = 1, Q becomes 00000000.

Accumulator_8bit_tb.v:

```
`timescale 1ns / 1ps

module accumulator_8bit_tb;

reg [7:0] A;
reg [7:0] B;
reg CLK;
reg RESET;

wire [7:0] Q;


// Unit under test
accumulator_8bit uut (
    .A(A),
    .B(B),
    .CLK(CLK),
    .RESET(RESET),
    .Q(Q)
);


// Clock
always #5 CLK = ~CLK;
```

```
initial begin

    // Initial condition and reset
    CLK = 0;
    RESET = 1;
    A = 8'b00000000;
    B = 8'b00000000;
    #10;


    // 3 + 5 = 8
    RESET = 0;
    A = 8'b00000011;
    B = 8'b00000101;
    #10;


    // 2 + 1 = 3
    A = 8'b00000010;
    B = 8'b00000001;
    #10;


    // 255 + 1 = 0 (8-bit overflow)
    A = 8'b11111111;
    B = 8'b00000001;
    #10;


    // 170 + 5 = 175
    A = 8'b10101010;
    B = 8'b00000101;
    #10;

    $finish;

end

endmodule
```

Output Analysis / Observations:

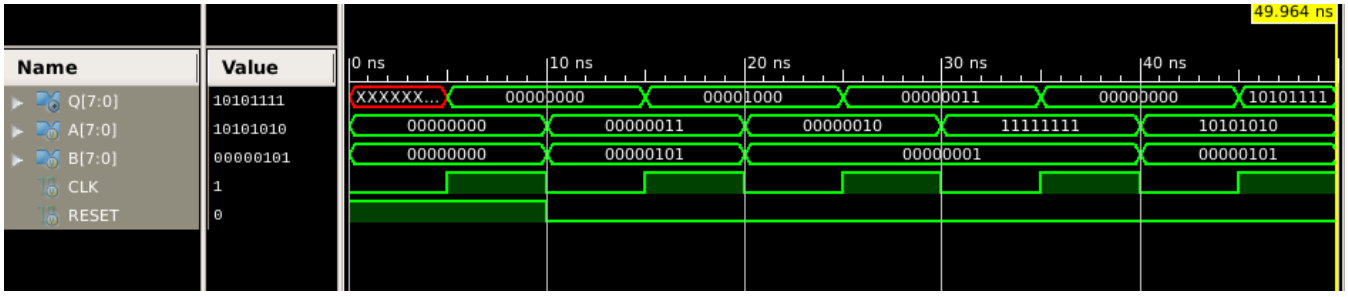


Fig. 2: Simulation Waveform of the 8-bit Accumulator

Table: Simulation Results of the 8-bit Accumulator:

Step	Operation	A	B	Expected Q
1	Reset	00000000	00000000	00000000
2	3 + 5	00000011	00000101	00001000
3	2 + 1	00000010	00000001	00000011
4	255 + 1	11111111	00000001	00000000
5	170 + 5	10101010	00000101	10101111

The simulated output Q matched the expected result for each applied input combination.

Github Repositories:

Repository Name: CSE450-VLSI-Lab
Project Folder: Midterm_8bit_Accumulator_Final
GitHub Link: https://github.com/naima-tabassum/CSE450-VLSI-Lab/tree/main/Midterm_8bit_Accumulator_Final

Challenges faced during the experiment

One challenge was understanding how the adder result is stored in Q at the rising edge of the clock. Another challenge was understanding the overflow case of 255 + 1, where the 8-bit output becomes 00000000.

Conclusion

In this experiment, the 8-bit accumulator was implemented by combining an 8-bit adder with an 8-bit register. The adder produced the sum of A and B, and the register stored the result at each rising edge of the clock. The RESET operation also worked correctly by clearing Q to 00000000. The required test cases, including the overflow case of 255 + 1, produced the expected outputs. Therefore, the operation of the 8-bit accumulator was verified through simulation.